WEEK 2 · LESSON 10 OF 24 · HANDOUT

Initialisation, Batch Norm and Vanishing Gradients

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=P6sfmUTpUmc — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Neural networks and backpropagation, step by step (building micrograd) — https://www.youtube.com/watch?v=VMj-3S1tku0

🎓 **Go deeper:** MIT 6.S191: Introduction to Deep Learning (2024) — https://www.youtube.com/watch?v=ErnWZxJovaM

Your goal in this lesson: Explain why gradients can vanish in deep networks, measure it, and fix it with ReLU, good initialisation, batch normalisation and residual connections.

Why deep networks were hard to train

Until about 2010, networks with more than a few layers barely learned. The reason is in backpropagation: the gradient for an early layer is a **product** of many local slopes, one per layer. The sigmoid's slope is at most 0.25. Multiply ten numbers of 0.25 or less and you get almost nothing — the early layers receive no learning signal. This is the **vanishing gradient** problem. (If the slopes are bigger than 1, gradients can **explode** instead.)

Measure it

We build a 10-layer network, run one backward pass, and print the average gradient size of each layer, from the first to the last:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.datasets import load_digits

digits = load_digits()
X = torch.tensor(digits.data[:200] / 16.0, dtype=torch.float32)
y = torch.tensor(digits.target[:200])

def deep_net(activation, he_init=False, batch_norm=False, depth=10):
    torch.manual_seed(0)
    layers = []
    for i in range(depth):
        linear = nn.Linear(64, 64)
        if he_init:
            nn.init.kaiming_normal_(linear.weight, nonlinearity="relu")
            nn.init.zeros_(linear.bias)
        layers.append(linear)
        if batch_norm:
            layers.append(nn.BatchNorm1d(64))
        layers.append(activation())
    layers.append(nn.Linear(64, 10))
    return nn.Sequential(*layers)

def gradient_sizes(model):
    F.cross_entropy(model(X), y).backward()
    return [f"{m.weight.grad.abs().mean().item():.0e}" for m in model if isinstance(m,
nn.Linear)]

print("sigmoid:         ", gradient_sizes(deep_net(nn.Sigmoid)))
print("ReLU + He init:  ", gradient_sizes(deep_net(nn.ReLU, he_init=True)))
print("sigmoid + BN:    ", gradient_sizes(deep_net(nn.Sigmoid, batch_norm=True)))
```

What you should see:

- **Sigmoid:** the first layer's gradients are around **0.00000000001** (1e-11), while the last layer's are around 0.01 — a difference of about a billion times. The early layers cannot learn.
- **ReLU with He initialisation:** every layer's gradient is about the same size (around 1e-3). Healthy.
- **Sigmoid with batch normalisation:** also healthy, even with the sigmoid.

Fix 1 — ReLU

ReLU's slope is exactly 1 for positive inputs, so it does not shrink the gradient as it passes through. That simple change is a big part of why "deep" learning took off.

Fix 2 — Good initialisation

Weights should start at a size that keeps signals from shrinking or growing from layer to layer:

- **He (Kaiming) initialisation** for ReLU: random normal with variance 2 ÷ (number of inputs).
- **Xavier (Glorot) initialisation** for tanh/sigmoid: variance about 1 ÷ (number of inputs).

PyTorch's layers already use sensible defaults; for very deep ReLU networks, `nn.init.kaiming_normal_` makes it explicit. This is also why you scaled weights by √(2/64) in the Week 1 project.

Fix 3 — Batch normalisation

Batch norm standardises each neuron's outputs over the current mini-batch (mean 0, standard deviation 1 — like StandardScaler, but inside the network), then lets the network learn its own scale and shift. It keeps every layer's inputs in a healthy range, allows bigger learning rates and often speeds training up a lot.

```
bn = nn.BatchNorm1d(3)
batch = torch.tensor([[1.0, 200.0, -5.0], [3.0, 400.0, -1.0], [5.0, 600.0, 3.0]])
bn.train()
print(bn(batch).detach().round(decimals=2))
```

Each column comes out with mean 0 and similar spread, whatever its original scale. Like dropout, batch norm behaves differently in `train()` and `eval()` mode: at prediction time it uses averages remembered from training.

Fix 4 — Residual connections

A **residual** (skip) connection adds a block's input to its output: output = x + block(x). The gradient can flow straight through the "+" — a highway past the layers. Residual networks (**ResNets**, 2015) made networks with 100+ layers trainable, and every transformer uses them too.

```
class ResidualBlock(nn.Module):
    def __init__(self, width):
        super().__init__()
        self.layers = nn.Sequential(nn.Linear(width, width), nn.ReLU(), nn.Linear(width,
width))

    def forward(self, x):
        return F.relu(x + self.layers(x))      # the skip connection

block = ResidualBlock(64)
print(block(torch.randn(5, 64)).shape)
```

Exploding gradients and clipping

When gradients explode (common in recurrent networks, Week 4), **gradient clipping** caps their size before the update:

```
model = deep_net(nn.ReLU, he_init=True)
F.cross_entropy(model(X), y).backward()
total = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
print("gradient size before clipping:", round(total.item(), 3))
```

Put it between `loss.backward()` and `optimizer.step()`.

Key terms

- **Vanishing gradient** — gradients shrink towards zero in early layers, so they stop learning.
- **Exploding gradient** — gradients grow huge and make training unstable.
- **He / Xavier initialisation** — starting weight sizes that keep signals healthy.
- **Batch normalisation** — standardising layer outputs over each mini-batch.
- **Residual connection** — adding a block's input to its output.
- **Gradient clipping** — capping the gradient size.

Remember this: Deep networks train well when gradients can flow: use ReLU, sensible initialisation, batch normalisation and skip connections, and clip gradients if they explode.

Name: _____ Date: _____

Activity

Task 1

Repeat the measurement with `nn.Tanh`. Is it better or worse than sigmoid?

Task 2

Use ReLU **without** He initialisation (the PyTorch default). How do the gradient sizes compare?

Task 3

Train the 10-layer sigmoid network with and without batch norm on the digits for 20 epochs. Compare validation accuracy.

Task 4

Stack four `ResidualBlock`s after an input layer and train them on the digits.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why do gradients vanish in deep sigmoid networks?
   A. The test set is small
   B. Many small slopes (at most 0.25) are multiplied together
   C. The data is too big
   D. ReLU is missing from the data
2. Why does ReLU help with vanishing gradients?
   A. Its slope is 1 for positive inputs, so it does not shrink the gradient
   B. It removes layers
   C. It adds noise
   D. It makes the network smaller
3. Which initialisation is designed for ReLU networks?
   A. Random text
   B. Zero initialisation
   C. He (Kaiming) initialisation
   D. All ones
4. What does batch normalisation do?
   A. Standardises layer outputs over each mini-batch
   B. Changes the labels
   C. Deletes small batches
   D. Sorts the data
5. What does a residual connection compute?
   A. block(x) only
   B. x × 0
   C. x + block(x)
   D. The loss
6. Where does gradient clipping go in the training step?
   A. Inside the dataset
   B. After saving the model
   C. Between loss.backward() and optimizer.step()
   D. Before the forward pass

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.